
SMM AI Department

Complete Technical Guide

NeuralForge Platform

7

Platforms

6

Sources

8

Niches

22

API Endpoints

Automated social media management powered by local LLMs.
Multi-source trend scanning, AI content generation, multi-platform publishing,
analytics tracking, and batch scheduling for 7 platforms.

Version 2.0 | March 2026

Self-hosted | Ollama + FastAPI

Tech: Python 3.12 | FastAPI | SQLite WAL | Ollama | ComfyUI | SearXNG

Table of Contents

1.

System Architecture

2.

API Endpoints Reference

3.

Database Schema

4.

Profile Management & Wizard

5.

Trend Scanning & Scoring

6.

Niche Routing Engine

7.

Platform Formats & Hashtag Manager

8.

Content Generation Pipeline

9.

Publishing & Publish Preview

10.

Batch Generation & Calendar

11.

Analytics & Event Log

12.

Token Health & Security

13.

Image Generation Pipeline

14.

Quick Reference

1. System Architecture

The SMM AI Department is a self-contained module within the NeuralForge platform. It provides end-to-end social media management: from trend discovery through AI content generation to multi-platform publishing and analytics tracking. The module is implemented as a FastAPI APIRouter mounted on the main NeuralForge server at localhost:9000.

Core Components

- FastAPI Router:

All SMM endpoints under /api/smm/* prefix, mounted via APIRouter. 22 endpoints organized into 8 functional groups.
- SQLite Database:

WAL-mode DB (smm_data.db) for queue, trends, analytics. Thread-local connections via threading.local() for safe concurrent access.
- JSON Profile Store:

Per-profile JSON files in smm_profiles/ directory. Profiles contain sensitive API tokens and are kept out of the database by design.
- Ollama LLM Backend:

Local inference via Ollama API at localhost:11434. Default model: qwen3.5:9b for generation, qwen3.5:72b for trend analysis.
- ComfyUI Integration:

Image generation through ComfyUI workflows at localhost:8188. Output directory: /home/user/Desktop/ComfyUI/output/
- SearXNG:

Self-hosted meta-search engine at localhost:8888. Used for web trend discovery across Google, Bing, and DuckDuckGo.

Data Directories

Directory	Purpose	Storage Type
smm_profiles/	Profile configurations with API credentials	JSON files
smm_trends/	Trend scan debug logs, publish logs	JSON files
smm_trends/_publish_log	Per-publish event logs for audit trail	JSON files
smm_queue/	Legacy queue storage (migrated to SQLite)	JSON (deprecated)
smm_images/	Generated post images and variants	PNG/JPG files
smm_data.db	Main database: queue, trends, analytics	SQLite WAL

Request Flow

The complete lifecycle of a social media post follows these stages:

- 1. Profile creation via wizard (4-step UI flow: basics, platforms, schedule, style)
- 2. Trend scan: multi-source aggregation with intelligent niche routing (6 source types)
- 3. Scoring: composite score from source type, engagement, freshness, keyword density
- 4. Content generation: LLM creates platform-specific posts from scored trend data
- 5. Image generation: optional ComfyUI workflow with LLM-generated prompts
- 6. Queue management: draft -> approved -> scheduled -> published lifecycle
- 7. Preview & editing: per-platform preview with inline editing and character limits
- 8. Publishing: simultaneous dispatch to all connected platforms with rate limiting
- 9. Analytics: periodic collection of engagement metrics (likes, comments, views, shares)
- 10. Cleanup: automated purge of old published items and analytics data

Module Dependencies

The SMM module registers its dependencies via `register_dependencies()` at startup. This function receives `load_modules_fn`, `start_module_fn`, and `stop_module_fn` from the main server, initializes the SQLite database, and performs a one-time migration of any existing JSON queue/trend files into SQLite.

Architecture note: Profiles are stored as JSON files (not in SQLite) because they contain sensitive API tokens. The `_smm_strip_credentials()` function masks all tokens before sending profile data to the frontend. Database migration from JSON to SQLite happens automatically on first startup if the queue table is empty and JSON files exist.

2. API Endpoints Reference

All endpoints are prefixed with `/api/smm/` and served by the FastAPI router. 22 endpoints organized into 8 functional groups.

Profile Management (5 endpoints)

Method	Endpoint	Description
GET	/api/smm/profiles	List all profiles (credentials stripped)
GET	/api/smm/profiles/{profile_id}	Get single profile by ID
POST	/api/smm/profiles	Create new profile (wizard step 1)
PUT	/api/smm/profiles/{profile_id}	Update profile (merges platforms safely)
DELETE	/api/smm/profiles/{profile_id}	Delete profile and its JSON file

Trend Scanning (4 endpoints)

Method	Endpoint	Description
POST	/api/smm/trends/scan	Start async trend scan for profile
GET	/api/smm/trends/history	List trend reports (id + timestamp)
GET	/api/smm/trends/by-id	Get specific trend report by ID
GET	/api/smm/trends/latest	Get latest trend report for profile

Content Generation (4 endpoints)

Method	Endpoint	Description
POST	/api/smm/generate	Generate platform-specific posts from topic
POST	/api/smm/generate-image-prompt	Generate image prompt from post text
POST	/api/smm/generate-image	Generate image via ComfyUI workflow
POST	/api/smm/regen-post	Regenerate single platform post

Queue, Publishing & Scheduling (7 endpoints)

Method	Endpoint	Description
GET	/api/smm/queue	List queue items for profile
POST	/api/smm/queue	Add item to queue
PUT	/api/smm/queue/{item_id}	Update queue item
DELETE	/api/smm/queue/{item_id}	Delete queue item
POST	/api/smm/publish	Publish queue item to platforms
POST	/api/smm/batch-generate	Batch generate posts for N days
GET	/api/smm/batch-status	Check batch generation progress

Calendar, Analytics & System (7 endpoints)

Method	Endpoint	Description
GET	/api/smm/calendar	Queue items grouped by date for calendar
GET	/api/smm/events	Recent publish events from logs
GET	/api/smm/analytics	Analytics summary for profile
GET	/api/smm/analytics/post/{queue_id}	Per-post analytics metrics
GET	/api/smm/token-health	Check token expiry for all platforms
POST	/api/smm/token-refresh	Refresh expired platform tokens
GET	/api/smm/storage	Storage usage statistics

Utility Endpoints (2 endpoints)

Method	Endpoint	Description
GET	/api/smm/image/{filename}	Serve generated image file
POST	/api/smm/cleanup	Delete published items older than N days
POST	/api/smm/github-search	Search GitHub repos by query

3. Database Schema

SQLite database (smm_data.db) with WAL journal mode for concurrent reads. Thread-local connections via threading.local(). Three tables with strategic indexing.

Table: queue

Stores all content queue items across their lifecycle (draft -> approved -> published).

Column	Type	Default	Description
id	TEXT PK	-	Unique slug-based ID (from profile name)
profile_id	TEXT NOT NULL	-	FK to profile JSON file
topic_title	TEXT	"	Topic/headline for the post
posts	TEXT	'{'	JSON: {platform: post_text} map
image	TEXT	NULL	Filename of generated image
image_variants	TEXT	'{'	JSON: per-platform image variants
status	TEXT	'draft'	draft approved published
scheduled_time	TEXT	NULL	ISO datetime for auto-publish
publish_results	TEXT	'{'	JSON: per-platform publish results
created	TEXT NOT NULL	-	ISO creation timestamp
updated	TEXT NOT NULL	-	ISO last-update timestamp

Indexes: idx_queue_profile (profile_id), idx_queue_status (status)

Table: trends

Stores LLM-analyzed trend reports from multi-source scans.

Column	Type	Default	Description
id	INTEGER PK	AUTO	Auto-increment primary key
profile_id	TEXT NOT NULL	-	Profile that ran the scan
timestamp	TEXT NOT NULL	-	When the scan was performed
report	TEXT NOT NULL	-	Full JSON trend report from LLM
created	TEXT NOT NULL	-	Row creation timestamp

Index: idx_trends_profile (profile_id)

Table: analytics

Hourly snapshots of engagement metrics per published post per platform.

Column	Type	Default	Description
id	INTEGER PK	AUTO	Auto-increment primary key
queue_id	TEXT NOT NULL	-	FK to queue item
profile_id	TEXT NOT NULL	-	Profile owner
platform	TEXT NOT NULL	-	Platform name (telegram, twitter, etc)
post_id	TEXT NOT NULL	-	Platform-specific post ID
likes	INTEGER	0	Like/reaction count
comments	INTEGER	0	Comment/reply count
views	INTEGER	0	View/impression count
shares	INTEGER	0	Share/repost count
collected_at	TEXT NOT NULL	-	Hour-rounded collection time

Indexes: idx_analytics_queue (queue_id), idx_analytics_profile (profile_id)

Deduplication: analytics_save() checks for existing entry in the same hour. If found, it UPDATES instead of INSERTing, preventing metric duplication on re-scans.

4. Profile Management & Wizard

Profiles are the central configuration unit. Each profile defines a brand identity, connected platforms, posting style, and schedule. Profiles are stored as JSON files in `smm_profiles/` directory to keep sensitive API tokens out of the database.

Profile Wizard (4-Step UI Flow)

The frontend wizard guides users through complete profile setup:

Step 1: Basics

- name:** Brand/project name (used to generate slug ID)
- niche:** Comma-separated keywords (e.g., 'AI, machine learning, devops')
- tone:** Content style: professional, casual, humorous, educational
- language:** ru, en, or ru+en for bilingual content

Step 2: Platforms

Connect up to 7 platforms with their API credentials. Each platform has its own configuration fields (`bot_token`, `channel`, `api_key`, etc). The UI shows connection status with green/red indicators.

Step 3: Schedule

- posting_schedule:** Per-day time slots for auto-publishing

The scheduler checks `queue_get_scheduled()` for approved items with past `scheduled_time` and auto-publishes them.

Step 4: Style

- visual_style:** Image generation preset: tech-dark, minimalist, vibrant, corporate
- hashtags:** Default hashtags added to all posts
- competitors:** Up to 3 competitor accounts for trend comparison

Profile JSON Structure

```
{
  "id": "neural-blog",
  "name": "Neural Blog",
  "niche": "AI, machine learning, LLM, devops",
  "tone": "professional",
  "language": "ru",
  "hashtags": ["#AI", "#ML", "#devops"],
  "competitors": ["@techblog"],
  "visual_style": "tech-dark",
  "platforms": {
    "telegram": {"enabled": true, "bot_token": "...", "channel": "@mychan"},
    "twitter": {"enabled": true, "api_key": "..."},
    "linkedin": {"enabled": true, "access_token": "..."}
  },
  "posting_schedule": {"mon": ["10:00", "18:00"], "wed": ["12:00"]}
}
```

Credential Security: `_smm_strip_credentials()`

Every API response that includes profile data passes through `_smm_strip_credentials()`. This function deep-copies the profile via JSON serialization and replaces all sensitive fields with '***' mask. The frontend never receives real tokens. Masked fields: `bot_token`, `webhook`, `api_key`, `api_secret`, `access_token`, `access_secret`, `page_token`, `person_urn`, `account_id`, `user_id`

A boolean 'connected' flag is set to True if any credential was present, allowing the UI to show connection status without exposing secrets.

*PUT /api/smm/profiles/{id} handles credential merging: if a field value is '***', the server skips it and keeps the existing token. This prevents UI save operations from wiping out real credentials.*

5. Trend Scanning & Scoring

The trend scanner aggregates content from 6 source types, scores each result using a multi-factor algorithm, then sends the top 35 results to a local LLM for analysis and topic extraction.

Data Sources (6 Types)

Source Type	Data Fetched	Engagement Metric
google_trends	Trending searches via RSS feed	Search volume (relative)
reddit	Hot posts from niche subreddits	Upvotes + comments
hackernews	Top/new stories from HN API	Points + comments
rss	RSS/Atom feed entries	N/A (content-based)
github	Trending repos + topic search	Stars + forks*3
searxng	Meta-search web results	Traffic estimates

Scoring Algorithm

Each collected result gets a composite score from 4 signal categories:

1. Source Type Boost (base credibility)

Source	Points	Reasoning
google_trends	+15	Highest: reflects real search demand
reddit	+12	Community-validated viral content
hackernews	+10	Tech-specific quality signal
rss	+8	Curated editorial content
github	+6	Code/project relevance
searxng/web	+0	Base level, no inherent boost

2. Engagement Boost (virality signal)

Engagement Level	Points	Threshold
>500	+25	Viral content (high upvotes, stars, etc)
>100	+15	Popular content
>30	+8	Moderate engagement
>0	+3	Minimal but non-zero engagement

3. Freshness & Relevance

Signal	Points	Condition
Published today	+20	freshness == 'day'
Keyword in title	+5	Each profile keyword found in title
Current year mentioned	+8	Year string found in content
Old content penalty	-15	Previous year or older found

4. Language Filters (hard penalties)

Filter	Points	Condition
CJK characters in title	-100	For ru/en profiles, removes CJK results
CJK domain for en profiles	-100	zhihu, baidu, csdn, bilibili domains

After scoring, results are sorted by `_score` descending. The top 35 are sent to the LLM (Ollama) as a structured prompt for analysis. The LLM returns a JSON report with topic suggestions, trend summaries, and content angles.

Source Implementation Details

Reddit Source (`_smm_src_reddit`)

Two-strategy approach for maximum reliability:

- Strategy 1 - SearXNG:** Search `'reddit {keyword}'` via SearXNG meta-search. Filters for `reddit.com/r/` URLs, excludes wiki pages. Up to 4 keywords, 12 results each.
- Strategy 2 - RSS Fallback:** Direct RSS feeds from `subreddit /hot/.rss` endpoints. Up to 3 subreddits, 8 entries each. 1-second delay between requests for rate limiting.

Cyrillic keywords are auto-translated to English for Reddit searches (e.g., `'restoran'` -> `'restaurant'`, `'fitnes'` -> `'fitness'`).

HackerNews Source (`_smm_src_hackernews`)

Uses HN Algolia API (hn.algolia.com/api/v1/search) with `points>15` filter. Searches up to 4 keywords, 12 hits per keyword. Engagement = points + comments.

GitHub Sources

Two GitHub data paths: trending page scraping and API search. The trending scraper parses github.com/trending HTML for repo links (up to 25 repos). The API search uses POST `/api/smm/github-search` with keyword-based queries. Auto-categorizes repos into 7 categories: Agent, Image Gen, RAG, Tool, Dataset, LLM, Chat.

Google Trends Source

Fetches Google Trends RSS feed for real-time trending searches. Provides the highest source type boost (+15) due to reflecting actual search demand.

6. Niche Routing Engine

SMM_NICHE_ROUTES maps 8 predefined niches to their optimal data sources and subreddit lists. The routing function `_smm_route_niche()` scores each route against the profile's niche keywords and selects the best match. If no match is found, a generic fallback (`searxng + reddit`) is used.

Niche: tech

Keywords: ai, ml, devops, programming, software, open source, llm, developer, kubernetes, docker, linux, python, api, agent, rag, lora, fine-tun, comfyui, stable diffusion

Sources: github_trending_ai, hackernews, reddit, github_trending, rss, searxng

Subreddits: MachineLearning, selfhosted, devops, LocalLLaMA, opensource, programming, StableDiffusion

RSS: hnrss.org/newest?points=50, techcrunch.com/feed/

Niche: crypto

Keywords: crypto, defi, web3, blockchain, bitcoin, ethereum, nft, krypto, bitcoin, blokchejn, token (+ Cyrillic variants)

Sources: reddit, rss, searxng

Subreddits: cryptocurrency, defi, ethereum, CryptoMarkets, Bitcoin

RSS: coindesk.com/arc/outboundfeeds/rss/

Niche: food

Keywords: restaurant, food, cooking, chef, recipe, cafe (+ Cyrillic: restoran, eda, retsept, kafe, kulinar)

Sources: reddit, rss, searxng

Subreddits: food, Cooking, FoodPorn, AskCulinary, MealPrepSunday

Site filters: tripadvisor.com, wolt.com, bolt.eu/food

Niche: fitness

Keywords: fitness, workout, health, gym, sport, running (+ Cyrillic)

Sources: reddit, google_trends, searxng

Subreddits: fitness, nutrition, bodyweightfitness, running, GYM

Niche: business

Keywords: saas, b2b, startup, marketing, product, growth, venture, investment

Sources: hackernews, reddit, rss, searxng

Subreddits: SaaS, startups, Entrepreneur, marketing, smallbusiness

RSS: `hnrss.org/newest?points=30&q=startup`

Niche: art

- Keywords:** art, design, comfyui, stable diffusion, flux, generative, midjourney, illustration
- Sources:** reddit, github_trending, searxng
- Subreddits:** StableDiffusion, comfyui, AIArt, generative, DigitalArt

Niche: gaming

- Keywords:** game, gaming, esports, steam, unity, unreal, gamedev
- Sources:** reddit, rss, searxng
- Subreddits:** gaming, Games, pcgaming, indiegaming, gamedev

Niche: education

- Keywords:** education, course, learn, tutorial, university, school, student
- Sources:** reddit, searxng, google_trends
- Subreddits:** learnprogramming, education, OnlineCourses

Geo-routing: `_smm_detect_locations()` scans niche keywords for city/country names and auto-adds geographic subreddits (e.g., `r/tallinn`, `r/berlin`). Cyrillic names are transliterated (e.g., `Moskva` -> `moscow`, `Sankt-Peterburg` -> `stpetersburg`).

7. Platform Formats & Hashtag Manager

SMM_PLATFORM_FORMATS defines the content constraints for each supported platform. These constraints are enforced during content generation (via LLM prompt) and post-generation (via trimming functions).

Platform Format Specifications

Platform	Max Chars	Max Tags	Content Style
Telegram	4096	5	Markdown, structured, emoji headings, source links
Twitter	280	3	Hook + key insight, 2-3 tags, provocative, no links in text
LinkedIn	3000	5	Professional tone, 3-5 paragraphs, expert position, CTA
Instagram	2200	28	Emotional storytelling, emoji-rich, 20-28 tags at end
Facebook	2000	5	Conversational, audience questions, engagement bait
Threads	500	5	Short opinion, conversational, 3-5 tags
Discord	2000	0	Casual community style, embed-friendly, NO hashtags

Hashtag Manager: `_smm_trim_hashtags()`

After LLM generation, each post passes through the hashtag trimmer. The function ensures platform limits are respected:

- If `max_hashtags == 0` (Discord): all `#hashtags` are stripped from the text
- If `hashtag count <= limit`: text passes through unchanged
- If `hashtag count > limit`: excess hashtags are removed from end of text

```
def _smm_trim_hashtags(text: str, platform: str) -> str:
    max_tags = SMM_PLATFORM_FORMATS.get(platform, {}).get('max_hashtags', 30)
    if max_tags == 0:
        return ' '.join(w for w in text.split() if not w.startswith('#'))
    words = text.split()
    hashtags = [w for w in words if w.startswith('#')]
    if len(hashtags) <= max_tags:
        return text
    excess = set(hashtags[max_tags:])
    return ' '.join(w for w in words if w not in excess)
```

LLM Prompt Integration

Platform formats are injected into the LLM generation prompt dynamically. For each enabled platform, a format string is built:

- telegram: up to 4096 chars. Markdown, structured, emoji headings...
- twitter: up to 280 chars. Hook + key insight...

The LLM is instructed to generate a JSON object with platform-keyed content that respects each platform's character limits and style guidelines.

8. Content Generation Pipeline

The /api/smm/generate endpoint creates platform-specific posts from a topic. It accepts a topic title, optional article text, profile ID, list of target platforms, and LLM model name.

Generation Flow

- 1. Load profile to get tone, language, hashtags, and enabled platforms
- 2. If article URL provided: scrape article text via _smm_scrape_url()
- 3. Build platform format instructions from SMM_PLATFORM_FORMATS
- 4. Construct LLM prompt with: topic, article context, profile tone/language, platform rules
- 5. Call Ollama API via _smm_call_ollama() with model and temperature settings
- 6. Parse JSON response via _smm_parse_json_obj() (3-stage fallback parser)
- 7. Trim hashtags per platform via _smm_trim_hashtags()
- 8. Return {platform: post_text} map

LLM Call Configuration

Parameter	Default	Description
model	qwen3.5:9b	Ollama model name
num_predict	8000	Max tokens to generate
temperature	0.7	Creativity vs consistency
think	True	Enable thinking/reasoning mode
timeout	360s	Request timeout with 1 retry

JSON Response Parser (_smm_parse_json_obj)

Three-stage fallback strategy for robust LLM output parsing:

- Stage 1:** Direct json.loads() on full response text

- Stage 2: Find last complete {...} block by scanning braces from end of string
- Stage 3: Fix truncated JSON: close unclosed quotes and braces, then parse

Post Regeneration

POST /api/smm/regen-post regenerates a single platform's post within an existing queue item. Takes queue_id, platform, and optional custom_prompt. Re-runs the LLM with the original topic context but allows the user to provide specific instructions for the regeneration.

9. Publishing & Publish Preview

The publishing system handles multi-platform dispatch with platform-specific APIs, image handling, and error tracking per platform.

Publish Flow (POST /api/smm/publish)

- 1. Load queue item and profile with credentials
- 2. For Instagram/Threads: pre-upload images to Imgur (Meta requires public URLs)
- 3. Rate-limit: 2-second delay between platform dispatches
- 4. Per platform: fetch credentials, trim hashtags, send via platform API
- 5. Collect results: {platform: {ok, message, post_id}}
- 6. Save publish log to smm_trends/_publish_logs/ as JSON
- 7. Update queue item status to 'published' with publish_results

Platform API Integrations

Platform	API Used	Image Support
Telegram	Bot API: sendMessage / sendPhoto	Direct file upload
Discord	Webhook URL: POST with embeds	Direct file upload
Twitter/X	OAuth 1.0a: POST /2/tweets	Media upload endpoint
LinkedIn	OAuth 2.0: ugcPosts API	Image via asset upload
Instagram	Graph API: media + publish	Imgur public URL required
Threads	Graph API: media + publish	Imgur public URL required
Facebook	Page API: /feed or /photos	Direct URL or upload

Publish Preview Modal

- Before publishing, the UI shows a preview modal displaying each platform's post in its rendered format. Users can:
- Review generated text for each platform side-by-side
 - See the attached image (or per-platform image variants)
 - Toggle individual platforms on/off for selective publishing
 - Edit post text inline before confirming publish
 - View character count vs platform limit with color-coded warnings

Image Variant System

Each queue item can have platform-specific image variants stored in image_variants JSON field. The _get_platform_image() function checks for a variant first, falling back to the base image. This allows generating square images for Instagram and landscape for Twitter from the same source.

Platform-Specific Publishing Details

Telegram Publishing

Uses Bot API with sendMessage (text-only) or sendPhoto (with image). Supports Markdown parse_mode for formatted text. The bot_token and channel ID are read from profile config. Direct file upload for images.

Instagram & Threads Publishing

Meta Graph API requires images to be accessible via public URL. The system pre-uploads images to Imgur before publishing. Separate uploads per platform to avoid Meta caching issues. 1-second delay between Imgur uploads for rate limiting. Two-step publish: first create media container, then publish it.

Discord Publishing

Uses webhook URLs (no bot token required). Sends content as embed-formatted messages. Direct file upload for images. No hashtags (max_hashtags=0, all # stripped).

Error Handling & Rate Limiting

2-second delay between platform dispatches to avoid triggering rate limits. Each platform result includes ok/fail status and error message. Failed platforms are logged in the event log for debugging. Publish results are saved to the queue item for status tracking.

10. Batch Generation & Calendar

Batch Generation (POST /api/smm/batch-generate)

Generates posts for multiple days at once. Runs as a background thread to avoid blocking the API. Maximum 14 days per batch.

Parameter	Type	Default	Description
profile_id	string	required	Profile to generate for
days	integer	7	Number of days (max 14)
model	string	qwen3.5:9b	Ollama model name
platforms	list	required	Target platforms list
generate_images	boolean	false	Also generate images via ComfyUI

Progress is tracked via _smm_batch_status dict and polled by GET /api/smm/batch-status. Status transitions: idle -> running (with progress message) -> done | error.

Calendar View (GET /api/smm/calendar)

Returns queue items grouped by date for the calendar UI component. Defaults to current week (Monday-Sunday) if no date range is specified.

```
GET /api/smm/calendar?profile_id=neural-blog&date_from=2026-03-23&date_to=2026-03-29
```

```
Response: {
  "ok": true,
  "days": {
    "2026-03-24": [
      {
        "id": "post-1",
        "topic_title": "AI Agents...",
        "status": "approved",
        "platforms": ["telegram", "twitter"],
        "scheduled_time": "2026-03-24T10:00"
      }
    ]
  }
}
```

Each calendar entry includes: id, truncated topic_title (50 chars), status, list of target platforms, scheduled_time, and image filename. The frontend renders this as a weekly grid with drag-and-drop rescheduling.

Auto-Scheduler

The auto-scheduler is a background task that periodically calls `queue_get_scheduled()`. It queries for items with `status='approved'` and `scheduled_time` in the past, then triggers automatic publishing. This enables the 'set and forget' workflow where users batch-generate a week of content and let the system publish on schedule.

11. Analytics & Event Log

Analytics System

Post-publish analytics tracking collects engagement metrics from each platform. Metrics are stored as hourly snapshots in the analytics table, enabling time-series analysis.

GET /api/smm/analytics: Aggregate summary for a profile (total likes, comments, views, shares, by-platform breakdown, top 5 posts)

GET /api/smm/analytics/post/{queue_id}: Per-post latest metrics across all platforms

analytics_save(): Upsert pattern - updates existing hour entry or inserts new one

analytics_cleanup_old(30): Purges analytics data older than 30 days

Analytics Summary Response

```
{
  "total_posts": 42,
  "total_likes": 1580,
  "total_comments": 234,
  "total_views": 28500,
  "total_shares": 89,
  "by_platform": {
    "telegram": {"likes": 800, "comments": 120, "views": 15000, "shares": 45},
    "twitter": {"likes": 400, "comments": 80, "views": 8000, "shares": 30}
  },
  "top_posts": [
    {
      "id": "post-1",
      "topic": "AI Agents Revolution",
      "likes": 150,
      "comments": 30,
      "views": 5200
    }
  ]
}
```

Event Log (GET /api/smm/events)

The event log tracks all publish operations. Each publish action writes a JSON log file to `smm_trends/_publish_logs/`. The `/api/smm/events` endpoint reads these logs and returns a summarized view.

- Each event includes: timestamp, topic title, success count, failure count, list of failed platforms
- Events are sorted by file modification time (newest first)
- Optional `profile_id` filter to show events for a specific profile
- Default limit: 20 most recent events

```
GET /api/smm/events?profile_id=neural-blog&limit=10
```

```
Response: {
  "events": [
    {
      "timestamp": "2026-03-25T14:30:00",
      "topic": "GPT-5 Release Analysis",
      "ok": 5,
      "fail": 1,
      "failed": ["instagram"],
      "total": 6
    }
  ]
}
```

Failure tracking enables automatic retry logic and helps identify platforms with expired tokens or misconfigured credentials.

12. Token Health & Security

Token Health Check (GET `/api/smm/token-health`)

Checks token expiry status for all platforms of a profile. Returns per-platform health status including days until expiry and whether a refresh is needed.

- Reads `token_expires_at` from each platform config
- Calculates days remaining and flags tokens expiring within 7 days
- Returns health map: `{platform: {status, days_remaining, refreshable}}`

Token Refresh (POST `/api/smm/token-refresh`)

Supports automatic token refresh for platforms with refresh token flows:

Threads Token Refresh

- Uses Meta Graph API: `/refresh_access_token` endpoint
- Extends long-lived token by 60 days
- Automatically updates token and expiry in profile JSON

LinkedIn Token Refresh

- Uses OAuth 2.0 `refresh_token` grant
- Requires `client_id` and `client_secret` in profile config
- Updates `access_token`, `refresh_token`, and expiry timestamp

Credential Security Model

Multi-layer security approach for API credentials:

Storage isolation: Credentials in JSON files, not in SQLite database

API stripping: `_smm_strip_credentials()` masks all tokens with `'***'` in responses

Merge protection: PUT endpoint skips `'***'` values to preserve real tokens

Path sanitization: `_smm_safe_id()` prevents directory traversal via regex filter

Input validation: Profile IDs cleaned with `[a-z0-9-_]` pattern only

Cleanup & Maintenance

Operation	Endpoint	Default	Description
Queue cleanup	POST /api/smm/cleanup	14 days	Delete published items older than N days
Analytics cleanup	analytics_cleanup_old()	30 days	Purge old analytics snapshots
Storage check	GET /api/smm/storage	-	Report disk usage for images, DB, profiles

13. Image Generation Pipeline

The image pipeline uses ComfyUI for AI image generation. Two-step process: first generate a prompt from post text via LLM, then execute ComfyUI workflow.

Step 1: Image Prompt Generation

POST /api/smm/generate-image-prompt takes the post text and generates a detailed image generation prompt via Ollama. The LLM is instructed to create a visual description suitable for Stable Diffusion / Flux models.

Step 2: ComfyUI Execution

POST /api/smm/generate-image sends the prompt to ComfyUI at localhost:8188. The workflow generates the image and saves it to the ComfyUI output directory. The server copies the result to `smm_images/` and links it to the queue item.

Image Serving

GET /api/smm/image/{filename} serves generated images directly via FastAPI FileResponse. Images are served from the `smm_images/` directory.

Visual Style Presets

The profile's `visual_style` setting influences image prompt generation. Available presets include: tech-dark (dark backgrounds, neon accents), minimalist (clean, whitespace), vibrant (colorful, high contrast), and corporate (professional, muted tones).

Image variants: Each platform can have a custom-sized variant (square for Instagram, landscape for Twitter/LinkedIn). Variants are stored in the queue item's `image_variants` field and served via the same `/api/smm/image/` endpoint.

GitHub Repository Categories

The GitHub search module auto-categorizes discovered repositories into 7 categories based on description and topic keywords. This helps trend analysis focus on relevant projects.

Category	Detection Keywords
Agent	agent, autonomous, crew, swarm, orchestrat, agentic
Image Gen	stable diffusion, comfyui, flux, image gen, diffusion, sdxl
RAG	rag, retrieval, vector, embedding, knowledge base
Tool	tool, cli, sdk, framework, library, api
Dataset	dataset, benchmark, eval, leaderboard
LLM	llm, language model, transformer, fine-tun, lora, inference
Chat	chat, assistant, copilot, companion

14. Quick Reference

Common Workflows

Create Profile and First Post

```
# 1. Create profile
POST /api/smm/profiles
{"name": "My Blog", "niche": "AI, tech", "tone": "professional",
 "language": "en", "platforms": {"telegram": {"enabled": true, "bot_token": "..."}}}

# 2. Scan trends
POST /api/smm/trends/scan
{"profile_id": "my-blog", "model": "qwen3.5:27b"}

# 3. Generate posts from topic
POST /api/smm/generate
{"profile_id": "my-blog", "topic": "AI Agents in 2026",
 "platforms": ["telegram", "twitter"], "model": "qwen3.5:9b"}

# 4. Publish
POST /api/smm/publish
{"queue_id": "post-id", "profile_id": "my-blog",
 "platforms": ["telegram", "twitter"]}
```

Batch Schedule a Week of Content

```
# Generate 7 days of posts with images
POST /api/smm/batch-generate
{"profile_id": "my-blog", "days": 7,
 "platforms": ["telegram", "twitter", "linkedin"],
 "generate_images": true, "model": "qwen3.5:9b"}

# Check progress
GET /api/smm/batch-status

# View calendar
GET /api/smm/calendar?profile_id=my-blog
```

Status Lifecycle

Status	Meaning	Transition
draft	Newly generated, not reviewed	User approves -> approved
approved	Reviewed and ready	Scheduler or manual -> published
published	Successfully sent to platforms	Cleanup after 14 days

Key Internal Functions

Function	Purpose
_smm_safe_id()	Sanitize profile/queue IDs (regex: [a-z0-9-_])
_smm_slugify()	Convert name to URL-safe slug
_smm_unique_id()	Generate unique slug, append UUID if collision
_smm_strip_credentials()	Deep-copy profile, mask all tokens with '***'
_smm_trim_hashtags()	Enforce per-platform hashtag limits
_smm_route_niche()	Score niche keywords against 8 route definitions
_smm_detect_locations()	Find city/country names for geo-subreddit routing
_smm_call_ollama()	Send prompt to Ollama with retry and response cleaning
_smm_parse_json_obj()	3-stage JSON parser for LLM output
_smm_scrape_url()	Extract article text from URL (strip HTML tags)
_has_cjk()	Detect CJK characters for language filtering

Environment Requirements

Service	URL	Required For
NeuralForge Server	localhost:9000	Main API server (FastAPI + Uvicorn)
Ollama	localhost:11434	LLM inference (trend analysis + content gen)
ComfyUI	localhost:8188	Image generation workflows
SearXNG	localhost:8888	Meta-search for trend discovery

Important: Never kill the NeuralForge server process directly. Use POST /api/restart endpoint or curl to trigger a graceful restart.